

AULA 03 - ALGORITMOS GENÉTICOS - FUNDAMENTOS

Algoritmo Genético (AG)

É um método de busca inspirado na **evolução da natureza**.

A matemática do AG copia o processo natural descritivo de Charles Darwin: adaptação ao meio ambiente.

Assim como os seres vivos evoluem para se adaptar ao ambiente, o AG evolui soluções para se adaptar ao problema.

Analogia:

- **Natureza:** Os animais mais fortes sobrevivem e têm filhotes
- **AG:** As melhores soluções são selecionadas e "reproduzem" melhores resultados

Os 4 Passos Principais

1. CRIAR → Uma população de soluções aleatórias
2. AVALIAR → Dar uma nota (fitness) para cada solução
3. SELECIONAR → Escolher as melhores para "reproduzir"
4. EVOLUIR → Gerar novas soluções (filhos) através de:
 - Cruzamento: misturar duas soluções
 - Mutação: fazer pequenas alterações

Repetir os passos 2, 3 e 4 até encontrar uma boa solução.

Fitness (A Nota da Solução) = quão boa é a solução

- Queremos **maximizar** o fitness (quanto maior, melhor)
- Se o problema é de minimização, transformamos (ex: fitness = -custo)

Representação

A Estrutura dos Dados (Representação)

A forma como "escrevemos" o problema no computador determina o tipo de cromossomo.

O que é um cromossomo? É a "receita" da solução.

Tabela os três mundos de aplicação:

- Binário: Utilizado em decisões do tipo "Sim ou Não" (ex: colocar ou não um arquivo/item na mochila).
- Permutação: Utilizado quando a ordem das coisas importa (ex: rota de entrega do Caixeiro Viajante, pacotes passando por roteadores).
- Números Reais: Utilizado para otimizar coordenadas contínuas x, y (como nas nossas funções 3D Rastrigin e Ackley).

Problema	Representação	Exemplo
Problema da Mochila	Binário (0=pegar, 1=não pegar)	[1, 0, 1, 1, 0] = pegar itens 1, 3 e 4
Caixeiro Viajante	Ordem de visita (permutação)	[3, 1, 4, 2, 0] = visitar cidade 3, depois 1, 4, 2 e 0
Otimizar parâmetros	Números reais	[2.5, -1.3, 0.7] = três parâmetros

Operadores Genéticos (Evoluir):

1 - Cruzamento (Crossover) - "Misturar" duas soluções - Recombinação

É o cruzamento genético. O filho recebe 50% das boas ideias do Pai 1 e 50% do Pai 2.

Exemplo com binário (ponto único):

Pai 1: 1 0 1 | 1 0 1

Pai 2: 0 1 0 | 1 1 0

↑ ponto de corte

Filho 1: 1 0 1 | 1 1 0 (parte do pai1 + parte do pai2)

Filho 2: 0 1 0 | 1 0 1 (parte do pai2 + parte do pai1)

2 - Mutação - "Mexer" um pouco a solução

A mutação ocorre com uma taxa muito baixa (1% a 5%). Ela serve para impedir que a população fique idêntica e presa num mínimo local (endogamia digital).

Exemplo com binário:

Original: 1 0 1 1 0 1

↓ muda um bit

Mutado: 1 0 1 0 0 1

Exemplo com permutação (swap):

Original: [1, 3, 5, 2, 4]

↓ troca duas posições

Mutado: [1, 2, 5, 3, 4]

Parâmetros Importantes

Parâmetro	O que faz	Valor típico
Tamanho da população	Quantas soluções por geração	20 a 100
Taxa de crossover	Chance de misturar soluções	80% a 90%
Taxa de mutação	Chance de alterar uma solução	1% a 5%
Gerações	Quantas vezes o AG vai repetir o ciclo (avaliar, selecionar, evoluir).	50 a 200

EXEMPLO PRÁTICO COM TODOS OS PARÂMETROS

CONFIGURAÇÃO PADRÃO (valor típico)

POPULACAO = 50 # 50 soluções por vez

GERACOES = 100 # 100 gerações

TAXA_CROSS = 0.85 # 85% de chance de crossover

TAXA_MUT = 0.02 # 2% de chance de mutação

PROBLEMA: OneMax (maximizar número de 1s)

Tamanho do cromossomo = 100 bits

Esperado:

→ Começa com ~50 uns (aleatório)

→ Depois de 50 gerações: ~85 uns

→ Depois de 100 gerações: ~98 uns

Exemplo: OneMax

Problema: Maximizar o número de 1s em uma string binária.

Objetivo: [1,1,1,1,1,1,1,1,1,1] ← 10 uns = fitness 10

Solução boa: [1,1,1,1,0,0,1,1,1,0] → fitness 7

Solução média: [1,0,1,0,1,0,1,0,1,0] → fitness 5

Solução ruim: [0,0,0,0,0,0,0,0,0,0] → fitness 0

O AG vai evoluir para encontrar a solução com 10 uns!